MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Sanjeev Khatri

Subject & Subject Code: **Semester:** OP using C++ - 3CS4-04 **Session:** 2023-24(Odd Sem.)

Unit No.: 1 **Date of Submission:**

Lect.No.	Topics Covered	Page No.	Total pages
1	Introduction : Introduction OOP, Procedural Vs. Object Oriented Programming	1-4	4
2	Principles of OOP, Benefits and applications of OOP, Overview	5-8	4
3	Program Structure, Namespace, Identifiers	9-10	2
4	Variables, Constants, Enum, Operators, Typecasting	11-14	4
5	Control structures, Array and Pointer	15-19	5
6	Abstraction Mechanism : Classes, Private, Public Constructors, Destructors	20-25	6
7	Member Data, Member Functions, Inline Function, Friend Functions	26-32	7
8	Static Members and References	33-37	5
9	Inheritance :Class Hierarchy, Derived Classes	38-44	7
10	Single Inheritance, Multiple, Multilevel, Hybrid Inheritance	45-55	11
11	Role of Virtual Base Class, Constructor and Destructor Execution	56-62	7
12	Base Initialization using Derived Class Constructors.	63-65	3

References Book:

1. Object Oriented Programming with C++ - E Balagurusamy
2. C++ : The Complete Reference – Herbert Schlitz
3. Object Oriented Programming in Turbo C++ - Robert Lafore.
4. C++ How to Program – Paul Dietel

05/10/2023

FACULTY **HOD** **DEAN ACADEMICS**

UNIT-1

LECTURE - 1

Introduction to Object Oriented Programming

Major Language Families-

1. **Machine Language** - Which is used by computer (CPU) as 0 and 1
2. **Assembly Language** - A low-level symbolic code converted by an assembler. It is also known as Low Level Programming Language.
3. **High Level Language** - Which is used by user and computer.

High Level Language-

1. **Procedural Language** - A procedural programming language includes functions, well organized procedures which helps the computer to provide desired output. It includes BASIC, C, FORTRAN, PASCAL, COBOL, etc.
2. **Object Oriented Programming Language (OOP)** – An Object Oriented Programming Language includes Encapsulation, Abstraction, Polymorphism, Inheritance, Class, Object. It includes C++, Java, Python, PHP, Scala, Perl, etc.

Programming Paradigm

Programming paradigms are the ways to classify programming languages based on their features. Not only programming languages but all fields like psychology, sociology and others use paradigms because every field have its own development age and every development has various different features.

Common Programming Paradigm-

- **Imperative** - In computer science, imperative programming is a programming paradigm of software that uses statements that change a program's state. In much the same way that the imperative mood in natural languages expresses commands, an imperative program consists of commands for the computer.
- **Declarative** - in which the programmer merely declares properties of the desired result, but not how to compute it.
- **Structured** - in which programmers works with clean, nested control structures.

MITRC

- **Procedural** - It has been derived from imperative programming, based on the concept of the procedure call. Procedures simply contain a series of computational steps to be carried out. Any given procedure might be called at any point during a program's execution.
- **Functional** - It is a programming paradigm where programs are constructed by applying and composing functions. It is a declarative programming paradigm in which function definitions are trees of expressions that map values to other values, rather than a sequence.
- **Object Oriented** – It is based on the concept of "objects", which can contain data and code: data in the form of fields, and code, in the form of procedures.

Imperative Programming Paradigm-

The learning process of OOPs will help to learn about-

In computer science, imperative programming is a programming paradigm of software that uses statements that change a program's state. In much the same way that the imperative mood in natural languages expresses commands, an imperative program consists of commands for the computer.

It performs step by step by changing state through assignment statements. It works as how to do.

Example-

Find Average of Three Numbers

```
int nums[3] = {3,4,5};
int total = 0;
float avg = 0.0;
for (int i=0;i<3;i++)
{
        total = total + nums[i];
}
avg = total / 3;
```

main focus of the program is to find out average.

MITRC

Declarative Programming Paradigm-

In which the programmer merely declares properties of the desired result, but not how to compute it.

It is divided as Logical, Functional and Database. It works as what to do.

Example-

select age from employeedata where age > 50

Structured Programming Paradigm-

Structured programming is a kind of imperative programming where control flow is defined by nested loops, conditions, etc. Main aim of this paradigm is to make programs easier.

Example-

```
if age > 30 then
        print("Age is above 30)
else if age >20 then
        print("Age is above 20)
else
        print("Age is less then or equals to 20)
end if
```

Procedural Programming Paradigm-

It has been derived from imperative programming, based on the concept of the procedure call. Procedures simply contain a series of computational steps to be carried out. Any given procedure might be called at any point during a program's execution. It is also known as top-down language.

Example-

```
int main()
{
        int n1, n2, n3, total;
        printf("Enter Three Values : ");
        scanf("%d %d %d", &n1, &n2, &n3);
```

MITRC

```
        total = n1 + n2 + n3;
        printf("Total :  %d", total);
        return 0;
}
```

Functional Programming Paradigm-

It is a programming paradigm where programs are constructed by applying and composing functions. It is a declarative programming paradigm in which function definitions are trees of expressions that map values to other values, rather than a sequence.

Example-

```
int a, b;
int z = add(a, b);
function int add(int x, int y)
{
        int total = x + y;
        return total;
}
```

Object Oriented Programming Paradigm-

It is based on the concept of "objects", which can contain data and code: data in the form of fields, and code, in the form of procedures. Objects contain data in the form of attributes and code in the form of methods.

Example-

```
int a, b;
int z = add(a, b);
function int add(int x, int y)
{
        int total = x + y;
        return total;
}
```

MITRC

LECTURE - 2

Principles of Object Oriented Programming

Characteristics of Object-Oriented Programming Languages-

Objects

Object is an instance of class. It is a piece of code which represents the real-life entity. When we define a class, no memory is allocated but when we create an object, memory is allocated. Each object contains data and code to manipulate the data. Without class object doesn't exists.

Example - Student

When we discuss about any student, we need his name, date of birth, class, marks, etc. which called Data.

After that we need to calculate total of marks, average marks, we need to create various functions like Total, Average, Output, etc. for reusability.

Class

Class is a collection of objects. It serves a blueprint of data and functions available in any object. It is a user-defined data type, which holds its own data members and member functions. Class is a structure where we can define variables, methods to utilize them by an object. Without object we can use static functions of class.

Example -

```
class Student
{
        name;
        dateofbirth;
        classname;
        marks1;
        marks2;
        marks3;
        total();
```

MITRC

```
        average();
        display();
}
```



Now we create object like - Student st;

Encapsulation

When we wrap up data and information under a single unit, it is known as encapsulation. It hides data for purpose of protection.

Example - College

It is an example of encapsulation where we have various departments like accounts, labs, student management, etc.

Data Abstraction

Abstraction is nothing but it hides implementation details while just presenting the features to the outside world. It refers to the act of representing essential features of any thing without including the background details or explanation. It reduce code complexity.

Example-

A real-life example is out Mobile Phone. We turn it on, make calls, click pictures, etc. but we do not know how all these thing works. How a phone gets connect with operators and other person, how it converts any click to a picture, etc.

Similarly, we call a function of a class in a program without knowing how it is working.

Polymorphism

Poly means many and morphism means forms means many forms. In other words, polymorphism is a Greek term that means the ability to take more than one form. Or we can say the ability of a message to be displayed in more than one form.

If something possesses different behaviour in different situations, this is known as polymorphism. Like a person can be father, husband, son, teacher at the same time.

MITRC

Dynamic Binding

Binding refers to the linking of a procedure call to the code to the executed in response to the call. Dynamic binding means the code associated with a given procedure call is not known until the time of the call at run-time. It is associated with a polymorphic reference depends upon the dynamic type of that reference.

Message Passing

An Object-Oriented program consists of a set of objects that communicate with each other.

A message for an object is a request for execution of a procedure and therefore will invoke a function (procedure) in the receiving object that generates the desired result. Message passing involves specifying the name of the object, the name of the function (message) and information to be sent.

Example-

Student ◄— Object

MarksAverage (Marks1, Marks2, Marks3)

Message Information

Inheritance

An Object-Oriented program consists of a set of objects that communicate with each other.

A message for an object is a request for execution of a procedure and therefore will invoke a function (procedure) in the receiving object that generates the desired result. Message passing involves specifying the name of the object, the name of the function (message) and information to be sent.

It is a mechanism of deriving a new class from an existing class.

- Single level Inheritance - A derived class is created from single base class like rose class derived from flower class (base class).
- Multi-Level Inheritance - A derived class is created from another derived classes and the same base class of another derived classes like World-India-Rajasthan-Alwar.
- Multiple Inheritance - A derived class is created from another derived class like Bird may be a parrot or a sparrow.
- Hierarchical Inheritance - A derived class is created from more than one base class like World-India-Delhi or Rajasthan and USA-New York or California.

MITRC

- Hybrid Inheritance - This is combination of more than one inheritance like hierarchical + multiple or multi-level + multiple.

Benefits of Object-Oriented Programming Languages-

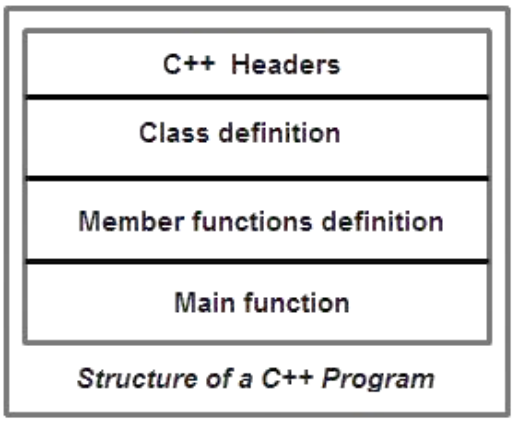
- Allows programmers to Develop Softwares.
- Through Inheritance, we can eliminate redundant code and extend the use of existing classes.
- Through Objects, we can easily partition our work in a project.
- It is possible to have multiple instances of an object to co-exist without any interference.
- Software complexity can be easily managed.
- Message passing techniques for communication between objects makes the interface descriptions with external systems much simpler.
- It overcome flaws of the procedural approach to programming such as Reusability, Maintainability.
- It can combine Data and Functions in a single unit.
- It can easily upgrade from small system to large system.
- It is simple to implement.

MITRC

LECTURE - 3

Program Structure, Namespace, Identifiers

A sequence of Instructions or Statements is known as Program. These statements form the structure of a C++ program. C++ program structure is contains various sections as Headers, Class Definition, Member Functions Definitions and Main Function.



Structure of a C++ Program

Headers

Headers are the name of predefined libraries having .h extension. These headers helps programmers to use predefined functions for smooth working. A few common headers are iostream.h, conio.h, string.h, math.h, etc.

Class Definition

A Class is a group of objects that includes common properties and relationships. Also, we can say in C++ a class is a new data type or user-defined data type that contains member variables and member functions that operates on the variables.

Member Function Definition

Member functions are operators and functions that are declared as members of a class. The definition of member function can be inside or outside of the class.

Main Function

Main functions is the gateway of any C++ Program. The main function serves as the starting point for program execution. It usually controls program execution by directing the calls to other functions in the program.

Example

```
#include <iostream>

using namespace std;

// main() is where program execution begins.

int main()

{

   cout << "Hello World" << endl;

   return 0;

}
```

Namespaces

A namespace is a declarative region that provides a scope to the identifiers (the names of types, functions, variables, etc) inside it. Namespaces are used to organize code into logical groups.

using namespace std;

The **using** statement informs the compiler that we want to use the std namespace. This is the namespace in which the entire Standard C++ library is declared. By using the **std namespace** we simplify access to the **standard** library.

Identifiers

An identifier is a sequence of characters used to denote one of the following-

- ❑ Object or Variable Name.
- ❑ Class, Structure, or Union name.
- ❑ Enumerated type Name.
- ❑ Member of a class, Structure, Union, or Enumeration.
- ❑ Function or Class-Member Function.

MITRC

LECTURE - 4

Variables, Constants, Operators, Typecasting

Variables

A variable is a name of memory location which is used to hold a value that may be modified during program execution. All variables must be declared before they can be used. The syntax to declare a variable is -

Type variable_list;

Type refers to a data type and variable_list may cinsist of one or more identifier names separated by commas.

In C++, there are different types of variables, like-

int - stores integers

double - stores floating point numbers

char - stores single character

string - stores text

bool - stores logical values true or false.

Variables that are declared inside a function are called Local Variables. If we declare a variable outside of any function, it is known as Global Variable. Local Variables work within the function and Global Variables work within whole program.

Constants

Constants refer to fixed values that the program cannot alter. If we declare a variable of type const, it becomes a Constant. Variables of type **const** may not be changed by our program.

Example

const int x=5;

A const variable will receive its value either from an explicit initialization.

MITRC

Operators

Operators are used to perform operations on Variables and Values. Also we can say they provide big assistance during coding and getting the desired output. C++ has a rich set of operators. We can divide operators list as per their use.

Arithmetic Operators – Used to perform Arithmetical Operations

+ Addition – To add two values

- Subtraction – Subtract one value from other

* Multiply – Multiplies two values

/ division – Divides one value by another

% Modulus – Returns the division remainder

++ Increment – Increases the value by 1

-- Decrement – Decreases the value by 1

Assignment Operator – Used to perform Assignment Opeations

= Assign values to variables

+= Addition Assignment

-= Subtraction Assignment

*= Multiplication Assignment

/= Division Assignment

%= Modulus Assignment

Comparison Operators – Used to Compare values.

== Equals to

!= Not Equal to

> Greater than

< Les then

>= Greater than or Equals to

<= Less than or Equals to

MITRC

Logical Operators – Used to perform Logical Operations

&& Logical and

|| Logical Or

! Logical Not

Type Casting

Type Casting refers to the conversion of one data type to another. It is also known as Type Conversion.

There are two types of type conversion in C++.

1. Implicit Conversion
2. Explicit Conversion (also known as Type Casting)

Implicit Type Conversion

The type conversion that is done automatically done by the compiler is known as implicit type conversion. This type of conversion is also known as automatic conversion.

Example-

```
#include <iostream>

using namespace std;

int main() {

        int a = 9;

        double b;

        b = a;

        cout << "a = " << a << endl;

        cout << "b = " << b << endl;

        return 0;

}
```

Explicit Conversion

When the we change data manually data from one type to another, this is known as explicit conversion. This type of conversion is also known as type casting.

There are three major ways in which we can use explicit conversion in C++. They are:

1. C-style type casting (also known as cast notation). Like-

MITRC

```
int a = 10;
float b;
b = (float)a;
```

2. Function notation (also known as old C++ style type casting)

```
int a = 10;
float b;
b = float(a);
```

3. Type conversion operators. It is a mixture of above two types. For Example-

```
#include <iostream>

using namespace std;

int main() {
   double a = 3.56;
   cout << "a = " << a << endl;
   // C-style conversion from double to int
   int b1 = (int)a;
   cout << "b1   = " << b << endl;

   // function-style conversion from double to int
   int b2 = int(a);
   cout << "b2   = " << b2 << endl;

   return 0;
}
```

MITRC

LECTURE - 5

Control Structures, Array and Pointer

Control Structures

Control Structures are used to create various logical, looping controls. C++ supports all basic control structures and implements them various control statements.

if statement

The if statement is implemented in three forms:

1. simple if statement

2. if… else statement

3. if… else if… statenebt

Simple if statement: It has only true part of a condition

```
if (condition)
{
        Action;
}
```

If.. else statement - It has both true and false parts of a condition

```
If (condition)
{
        Statment1
}
Else
{
        Statement2
}
```

If.. else if.. statement - It allows to check nested conditions

If (condition) → Condition One

MITRC

```
{
        Statment1
}
else if → condition two
{
        Statement2
}
```

switch-case statement

This is a multiple-branching statement which is based on a condition, the control is transferred to one of the many possible points;

```
switch(expr)
{
        case 1:
                action1;
                break;
        case 2:
                action2;
                break;
        .. ..
        default:
                message
}
```

while statement → It is a looping statement and works when a condition is true

```
While(condition)
{
        Stements
```

MITRC

}

do-while statement → It is a looping statement and It works first and then check condition

do

{

Stements

} while(condition);

For-loop statement →It is a looping statement and perform a repeation based on condition.

for(initialization;comparison;increament/decrement)

{

Statements;

Statements;

}

Array

An array is a series of elements of the same type placed in contiguous memory locations that can be individually referenced by adding an index to a unique identifier. Like a regular variable, an array must be declared before it is used. A typical declaration for an array in C++ is:

type name [elements];

Example

int rollno [5] = { 161, 112, 277, 440, 2071 };

It is an integer type array of five elements and will work as rollno[0], rollno[1], rollno[2], rollno[3], rollno[4].

The above approach of array is known as single dimension array.

MITRC

Multidimensional arrays

Multidimensional arrays can be described as "arrays of arrays.

type name [elements][elements];

Example

int rollno [3][4];

Arrays as parameters

At some moment we may need to pass an array to a function as a parameter. In order to accept arrays as parameters the only thing that we have to do when declaring the function is to specify in its parameters the element type of the array, an identifier and a pair of void brackets []. For example, the following function:

void procedure (int arg[])

Pointer

The variable that stores the reference to another variable is what we call a pointer. Variables contain values of a specific data type and Pointers contain memory address of the variables. Using a pointer we can directly access the value stored in the variable which it points to. To do this, we simply have to precede the pointer's identifier with an asterisk (*), which acts as dereference operator and that can be literally translated to value pointed by.

Syntax-

type * name;

type is the data type of the value that the pointer is intended to point to.

Example-

```
#include <iostream>
using namespace std;
int main ()
{
  int numbers[5];
  int * p;
  p = numbers; *p = 10;
  p++; *p = 20;
```


MITRC

```
    p = &numbers[2]; *p = 30;
    p = numbers + 3; *p = 40;
    p = numbers; *(p+4) = 50;
    for (int n=0; n<5; n++)
        cout << numbers[n] << ", ";

    for (int a=0; a<5; a++)
    {
        cout <<"\n"<< *p;
        p++;
    }

    return 0;
}
```

MITRC

LECTURE - 6

Abstraction Mechanism : Classes

Abstraction is a mechanism to hide irrelevant details and represent only the essential features so that one can focus on important things at a time. The classes in OOP are defined in such a way that the data is hidden from the outside world and the functions form the public interface.

Classes

Class is a group of objects that includes common properties and relationships. Also, we can say in C++, a class is a new data type or user-defined data type that contains member variables and member functions that operates on the variables.

Example

Class of Cars. All Cars can run, they have engine, tires, steering, etc. There are many different cars with different names or makes but all they have same features.

Generally, a class specification has two parts-

Class Declaration

Class Function Definition

Class declaration describes the type and scope of its members. The class function definition describes how the class functions are implemented.

Syntax -

```
class class-name
{
        private:
                variable declarations;
                function declarations;
        public:
                variable declarations;
                function declarations;
```

MITRC

}

The members declared as private can be accessed from within the class only, but public members can be accessed from outside the class also.

Object

After creating a class, we can create variables of that type by using the class name. Objects communicate by sending message to each other when a program is being executed. Each object required data and function call to execute and modify the data.

We can create objects in a similar way as we declare variables.

Syntax –

class-name object-name;

The class name is the name of the class which has been called to execute its data and functions. The object name is a variable name that represents the class.

This process of creating an object from a class is known as instantiation.

For **Example** –

Student student1

Student student2

Here is, Student is the name of a class which has data and functions.

And student1, student2 are the variable names who are representing the Student Class.

```
Class Student
{
        int rn;
        void show()
        {
                function body;
        }
}
```

MITRC

```
Int main()
{
        Student student1; ←—— Using class with a variable name student1.
        student1.show();  ←—— Using function available in Student Class.
}
```

Example -

```
class student
{
   public:
      int rlno;
      char name;
};

int main()
{
   student student1;
   student student2;
   student1.rlno = 1;
   student1.name = 'ABC';
   student2.rlno = 2;
   student2.name = 'BCD';

   cout << "Roll of Student-1 is : " << student1.rlno << "\n";
   cout << "Name of Student-1 is : " << student1.name << "\n";
   cout << "Roll of Student-2 is : " << student2.rlno << "\n";
   cout << "Name of Student-2 is : " << student2.name << "\n";
   return 0;
```

MITRC

}

Constructors

To customize how a class initializes its members, or to invoke functions when an object of our class is created, define a constructor. A constructor has the same name as the class and no return value, followed by parentheses ().

Example-

Inside the class-

```
class student
{
   public:
      student(int rollno, string name); →Constructor declaration
};
```

Outside the class – to do this we use scope resolution :: operator.

```
class student
{
}
student::student(int rollno, string name)
{
}
```

Typically, constructors have public accessibility so that code outside the class definition or inheritance hierarchy can create objects of the class. The public keyword is an access specifier.

Access specifiers

Access specifiers define how the members (attributes and methods) of a class can be accessed. In C++, there are three access specifiers:

public - members are accessible from outside the class

private - members cannot be accessed (or viewed) from outside the class

MITRC

protected - members cannot be accessed from outside the class, however, they can be accessed in inherited classes.

Example-

```
class student {
  public:    // Public access specifier
    int rn;
  private:   // Private access specifier
    int marks1;
};

int main() {
  student stdnt;
  stdnt.rn = 50;  // public
  stdnt.marks1 = 70;  // Not allowed (private)
  return 0;
}
```

With the help of these access specifiers, we can create Constructors public or private.

Destructors

Destructors are usually used to deallocate memory and do other clean up for a class object and its class members when the object is destroyed. A destructor is called for a class object when that object passes out of scope or is explicitly deleted.

A destructor is a member function with the same name as its class prefixed by a ~ (tilde).

```
class X {
public:
  // Constructor for class X
  X();
```

MITRC

```
 // Destructor for class X
 ~X();
};
```

A destructor takes no arguments and has no return type.

MITRC

LECTURE - 7

Member Data, Member Functions

Data Members

The variables which we declare in any class by using any basic data type like int, float, char, etc. or derived data type like class, structure, pointer, etc. are known as Data Members.

Any Data member declared in a class can be private, public or protected. Data members declared as public can be accessed directly from anywhere in the program, but private and protected variables cannot be accessed directly. To access, use or initialize the private and protected data members, we will have to create special public member functions.

Public Data Members

To show how to initialize and use the public data members using the dot(.) operator and respective object of class.

Example-

```
class Student
{
   public:
   int rollno;
   string name;
};

int main()
{
   Student S1;
   Student S2;

   // setting values for S1 object
   S1.rollno=11;
   S1.name="ABC";
```

MITRC

```
   // setting values for S2 object
   S2.rollno=22;
   S2.name="BCD";

   cout <<"Name and Roll no of S1 is: "<< S1.name << "-" << S1.rollno;
   cout <<"Name and Roll no of S2 is: "<< S2.name << "-" << S2.rollno;
}
```

Private Data Members

To access, use and initialize the private data member you need to create getter and setter functions, to get and set the value of the data member. The setter function will set the value passed as argument to the private data member, and the getter function will return the value of the private data member to be used. Both getter and setter function must be defined public.

Example-

```
class Student
{
   private:    // private data member
   int rollno;

   public:
   // public function to get value of rollno - getter
   int getRollno()
   {
      return rollno;
   }
   // public function to set value for rollno - setter
   void setRollno(int i)
   {
```

MITRC

```
    rollno=i;
  }
};

int main()
{
  Student A;
  A.rollono=1;  //Compile time error
  cout<< A.rollno; //Compile time error
  A.setRollno(1);  //Rollno initialized to 1
  cout<< A.getRollno(); //Output will be 1
}
```

Member functions are operators and functions that are declared as members of a class. The definition of member function can be inside or outside of the class.

If we declare a member function outside the class, then we must use scope resolution :: operator along with the class name.

Example- showing definition of function outside the class;

```
class Studentmarks
{
  public:
    int sum(int subject1, int subject2, int subject3);
};

int Studentmarks :: sum(int subject1, int subject2, int subject3)
{
  int total = subject1 + subject2 + subject3;
  return total;
}
```


MITRC

```
void main()
{
   int sub1 = 5;
   int sub2 = 6;
   int sub3 = 7;
   Studentmarks sm1;
   int total  = sm1.sum(sub1, sub2, sub3);
   cout << "Total Marks : " << total;
}
```

If we declare a member function inside the class, it can be defined directly and can be used with dot(.) operator.

Example- showing definition of function inside the class;

```
class Studentmarks
{
   public:
      int sum(int subject1, int subject2, int subject3)
      {
         int total = subject1 + subject2 + subject3;
         return total;
      }
};


void main()
{
   int sub1 = 5;
   int sub2 = 6;
   int sub3 = 7;
   Studentmarks sm1;
```

MITRC

```
    int total  = sm1.sum(sub1, sub2, sub3);
    cout << "Total Marks : " << total << endl;
}
```

Types of Member Function-

1. Simple Functions
2. Inline Functions
3. Friend Functions
4. Static Functions

Simple Function

These are the basic member functions, which does not have any special keyword like static as prefix.

Example- showing simple function

```
int sum(int subject1, int subject2, int subject3)
{
    int total = subject1 + subject2 + subject3;
    return total;
}
```

Inline Function

An inline function is a function that is expanded in a line when it is called. When an inline function is called, the whole code of the inline function gets inserted.

Inline functions are used to save memory space. When we call a normal function, it takes a lot of time to execute but inline functions can be used directly. We do not need to call a function. Inline function can be used for small calculations, small processing to save execution time.

Example-

```
inline int add(int val1, int val2)
{
    return(val1 + val2);
```

MITRC

```
}

int main()
{
   int c = add(2,3);
   return 0;
}
```

Friend Function

If a function is defined as friend function, the protected and private data of a class can be access using the function. By using the keyword friend, compiler knows the given function is a friend function.

To access the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

The friend function can be defined anywhere in the program like a normal C++ function. The function definition does not use either the keyword friend or scope resolution operator.

Example-

```
class Box
{
   private:
      int length;
   public:
      Box(): length(0) { }
      friend int printLength(Box); //friend function
};

int printLength(Box b)
{
  b.length += 10;
   return b.length;
```

MITRC

```
}

int main()
{
   Box b;
   cout<<"Length of box: "<< printLength(b) << endl;
   return 0;
}
```

Characteristics of member functions

- A program can have several classes and they can have member functions with the same name, ambiguity is resolved using the scope resolution (::) operator.
- The same function can be used in any number of classes.
- The private data or private function can be accessed by public member function.
- The member function can have access of one another without using any object or dot(.) operator.
- Member function defined can be defined as public act as an interface between the service provider and the service seeker.
- Member functions can take default arguments as well.

MITRC

LECTURE - 8

Static Members and References

Static Data Members

Class members can be declared using the storage class specifier static in the class member list. Only one copy of the static member is shared by all objects of a class in a program.

- It is initialized to zero when the first object of its class is created. No other initialization is permitted.
- Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
- It is visible only with in the class, but its lifetime is the entire program. Static variables are normally used to maintain values common to the entire class. For example, a static data member can be used as a counter that records the occurrence of all the objects.

When we declare an object of a class having a static member, the static member is not part of the class object. We access a static member by qualifying the class name using the :: (scope resolution) operator.

In the following example, we can refer to the static member f() of class type X as X::f() even if no object of type X is ever declared:

Example-

```
#include<iostream>
using namespace std;
class numcounter
{
   static int counter; //count is static
   int num;
   public:
      void getdata(int a)
      {
         num=a;
         counter++;
      }
      void getcount(void)
```

MITRC

```
    {
       cout <<  "count : ";
       cout <<  counter << endl;
    }
};

int numcounter :: counter = 5; //count defined
int main( )
{
   numcounter a1,a2,a3;
   a1.getdata(5);
   a1.getcount( );
   a2.getdata(6);
   a2.getcount( );
   a3.getdata(7);
   a3.getcount( );
   a1.getcount( );
   return 0;
}
```

Static Function

These functions work for the class as whole rather than for a particular object of a class. Static is a keyword which can be used with data members as well as the member functions. A function is made static by using static keyword with the function name.

It can be called using the object and the direct member access operator. But it is more typical to call a static member function by itself, using class name and scope resolution :: operator.

Example-1

```
class abc
{
        public:
```


MITRC

```
        static void bcd();
};
int main()
{
    abc :: bcd(); //calling member function directly with class name;
}
```

Example-2

```
class Car
{
  public:
    // declare a static data member
    static int staticmember;
    Car()
    {
      static_member++;
    }
};

// initialized the static data member to 0
int Car :: static_member = 0;
 int main ()
{
  // create object for the class Car
  Car c1;
  //create another object
  Car c2;
  cout << " \n No. of objects created in the class: " << Car :: staticmember << endl;
```

MITRC

```
    return 0;
}
```

References

We all familiar with variables and pointers. Variables contain values of a specific data type and Pointers contain memory address of the variables. A pointer is also a variable name, but it works as an alias for any other variable. In C++ we use another way to write a variable name is Reference.

In the same way as pointers, we use References in C++. A reference is also a variable name for a variable which hold the memory address of that variable. A reference can be created by using an ampersand (&) operator before a variable name just like we use astric (*) with pointer name.

Suppose we declare an integer as - int x=5, then we can create a reference for this variable as - int &ref=x.

Example-

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
   int y=50;
   int& r=y;
   cout << r << endl;
   getch();
   return 0;
}
```

Output : 50

Example showing use of Reference and Pointer together-

```
#include <iostream>
```

MITRC

```
#include <conio.h>
using namespace std;
int main()
{
   int* pntr;
   int y=50;
   int& r=y;
   pntr = &y;
   cout << r << endl;
   cout << *pntr;
   getch();
   return 0;
}
```

Output :

50

50

Difference between Reference and Pointer

Initialization - Pointers can be initialized any time. But Reference must be initialized when it is created.

Reinitialization - Pointers can be reinitialized any number of times. But References cannot reinitialize.

Values - Pointers can be null. But Reference cannot be null.

MITRC

LECTURE - 9

Inheritance : Class Hierarchy, Derived Classes

Inheritance

Inheritance is the process by which one object can acquire the properties of another object. This is important because it supports the concept of hierarchical classification. Inheritance is one of the most important features of Object-Oriented Programming.

Inheritance is a process in which, new classes are created from the existing classes. The new class created is called as **derived class** or **child class** and the existing class is known as the **base class** or **parent class**. The derived class now is said to be inherited from the base class.

The general form for inheritance is shown here-

class derived-class : <access-specifier> base-class { // body of derived class }

In this-

class - keyword to create a new class

derived-class - name of the new class, which will inherit the base class

access-specifier - either of private, public, or protected. If neither is specified, PRIVATE is taken as default.

base-class - name of the base class.

Example-

```
#include<iostream>
using namespace std;

class onestd
{
   int rno;
   char stname[100];

   public:
      void set_std()
```


MITRC

```
    {
        cout<<"Enter the Roll Number:";
        cin>>rno;
        fflush(stdin); //fflush(stdin) is used to flush or clear the output buffer of the stream
        cout<<"Enter the Name:";
        cin.get(stname,100);
    }

    void display_name()
    {
        cout<<endl<<rno<<"\t"<<stname<<"\t";
    }
};

class student: private onestd
{
  char course[50];
  int marks;

  public:
  void set_student()
    {
        set_std();
        cout<<"Enter the Course Name:";
        fflush(stdin); //fflush(stdin) is used to flush or clear the output buffer of the stream
        cin.getline(course,50);
        cout<<"Enter Marks:";
        cin>>marks;
```

MITRC

```
    }

    void display_std()
    {
      display_name();
      cout<<course<<"\t"<<marks<<endl;
    }
};

main()
{
  student stds;
  stds.set_student();
  stds.display_std();
  return 0;
}
```

Hierarchical Inheritance

When more than one classes are derived from a single base class, such inheritance is known as Hierarchical Inheritance, where features that are common in lower level are included in parent class. Problems where hierarchy must be maintained can be solved easily using this inheritance.

Syntax of Hierarchical Inheritance

```
class base_classname
{
  properties;
  methods;
};
```

MITRC

```
class derived_class1:visibility_mode base_classname
{
   properties;
   methods;
};

class derived_class2:visibility_mode base_classname
{
   properties;
   methods;
};
class derived_classN:visibility_mode base_classname
{
   properties;
   methods;
};
```

Example-

```
#include <iostream>
#include <conio.h>
#include <cstdio>

using namespace std;

class person
{
   char fullname[100], gender[10];
   int age;
```

MITRC

```
   public:
      void getdata()
      {
         cout<<"Name: ";
         fflush(stdin);  /*clears input stream*/
         cin.get(fullname, 100);
         cout<<"Age: ";
         cin>>age;
         cout<<"Gender: ";
         cin.get(gender, 10);
      }
      void display()
      {
         cout<<"Name: "<<fullname<<endl;
         cout<<"Age: "<<age<<endl;
         cout<<"Gender: "<<gender<<endl;
      }
};

class student: public person
{
   char institute[100], level[20];
   public:
      void getdata()
      {
         person::getdata();
         cout<<"Name of College/School: ";
         fflush(stdin);
```

MITRC

```
        cin.get(institute, 100);
        cout<<"Level: ";
        cin.get(level, 20);
    }
    void display()
    {
        person::display();
        cout<<"Name of College/School: "<<institute<<endl;
        cout<<"Level: "<<level<<endl;
    }
};

class employee: public person
{
  char company[100];
  float salary;
  public:
    void getdata()
    {
        person::getdata();
        cout<<"Name of Company: ";
        fflush(stdin);
        cin.get(company, 100);
        cout<<"Salary: Rs.";
        cin>>salary;
    }
    void display()
    {
```

MITRC

```
        person::display();
        cout<<"Name of Company: "<<company<<endl;
        cout<<"Salary: Rs."<<salary<<endl;
    }
};

int main()
{
   student s;
   employee e;
   cout<<"Student"<<endl;
   cout<<"Enter data"<<endl;
   s.getdata();
   cout<<endl<<"Displaying data"<<endl;
   s.display();
   cout<<endl<<"Employee"<<endl;
   cout<<"Enter data"<<endl;
   e.getdata();
   cout<<endl<<"Displaying data"<<endl;
   e.display();
   getch();
   return 0;
}
```

MITRC

LECTURE - 10

Single Inheritance, Multiple, Multilevel, Hybrid Inheritance

Single Inheritance

When a class inherits from a single base class, it is known as single inheritance.

Example-

```
#include<iostream>
#include<conio.h>
#include<string.h>
using namespace std;
class staff
{
   int age;
   char name [10];
   public:
      void get();
      void show();
};

void staff::get()
{
   cout <<"Enter Name : ";
   cin >> name;
   cout << "Enter Age : " ;
   cin >> age;
};

void staff::show()
{
```

MITRC

```
   cout <<"Name is : "<< name <<"\n" << "Age :"<< age;
};

class officer:public staff //derived class (publicly)
{
   int now;
   public:
      void get();
      void show();
};

void officer::get()
{
   staff::get(); //the calling of base class input function.
   cout << "Total Staff Working with you : ";
   cin >> now;
};

void officer::show()
{
   staff::show(); //calling of base class output function.
   cout <<"No. of staff are: " << now <<"\n";
};

void main()
{
   staff W1;
   officer M1;
```

MITRC

```
   M1.get();
   M1.show() ;
}
```

Multiple Inheritance

A class can inherit the attributes of two or more classes. This mechanism is known as **MULTIPLE INHERITENCE**. Multiple inheritance allows us to combine the features of several existing classes as a starting point for defining new classes.

```
#include<iostream>
#include<conio.h>
using namespace std;
class father //Declaration of base class
{
   int age ;
   char name[20] ;
   public:
      void get();
      void show();
};
void father::get()
{
   cout << "Enter Father's Name : ";
   cin >> name;
   cout << "Enter Father's Age : ";
   cin >> age;
};

void father::show()
{
   cout<<"Father's Name : "<<name<<"\n";
```

MITRC

```
    cout<<"Father's age : "<<age<<"\n";
};

class mother //Declaration of base class 2
{
    char name [20] ;
    int age ;
    public:
    void get ( )
    {
        cout << "Enter Mother's Name : ";
        cin >> name;
        cout << "Enter Mother's Age : ";
        cin >> age;
    }
    void show ( )
    {
        cout << "Mother's Name : "<<name<<"\n";
        cout << "Mother's Age : "<<age<<"\n";
    }
};

class daughter : public father, public mother //derived class inheriting
{
    char name [20] ; //the features of both the base class
    int age;
    public:
    void get ( ) ;
```

MITRC

```
    void show ( ) ;
};

void daughter::get()
{
    father::get() ;
    mother::get() ;
    cout << "Enter Child's Name : ";
    cin >> name;
    cout << "Enter Child's age : ";
    cin >> age;
};

void daughter::show()
{
    father::show();
    mother::show ( ) ;
    cout << "Child's Name : " <<name<<"\n";
    cout << "Child's age : " <<age<<"\n";
}
void main ( )
{
    daughter d1;
    d1.get ( ) ;
    d1.show ( ) ;
}
```

MITRC

Multilevel Inheritance

When class A serves as a base class for a derived class B which in turn serves as a base class for the derived class C, it is called Multilevel Inheritance. In this case B also called as Intermediate base class.

Example-

```
#include<iostream>
#include<conio.h>
using namespace std;
class staff // Base class declaration
{
   int age;
   char name [20];
   public:
      void get( );
      void show( );
};

void staff::get()
{
   cout << "Enter Name : ";
   cin >> name;
   cout << "Enter Age : ";
   cin >> age;
};

void staff::show()
{
   cout << "Name is : " <<name;
   cout<< "\n" << "Age : " <<age;
```

MITRC

```
    cout << "\n";
};

class officer:public staff //Intermediate base class derived
{ //publicly from the base class
    int now;
    public:
        void get();
        void show();
};

void officer::get()
{
    staff::get() ; //calling get() function of base class
    cout << "Enter Staff count under Officer : ";
    cin >> now;
};

void officer::show()
{
    staff::show() ; //calling show() function of base class
    cout << "No. of staff under Officer are : "<< now;
    cout << "\n";
};

class owner:public officer //declaration of derived class
{ //publicly inherited from the
    int now; //intermediate base class
```

MITRC

```
   public:
      void get();
      void show();
};

void owner::get()
{
   officer::get();
   cout << "Enter No. of staff under Officer are : ";
   cin >> now;
   cout << "\n";
};

void owner::show()
{
   officer::show();
   cout << "Total No. of Staff under Officer are : ";
   cout << now;
   cout << "\n";
};

void main ( )
{
   owner cl ;
   cl.get() ; cout << "\n";
   cl.show() ;
}
```

MITRC

Hybrid Inheritance

Sometimes, we need to apply two or more types of inheritance to design a program. To do this Hybrid Inheritance, which is the combination of one or more types of the inheritance.

Example-

```
#include<iostream>
#include<conio.h>
using namespace std;
class student //base class declaration
{
   protected:
   int r_no;
   public:
   void get_n(int a)
   {
      r_no =a;
   }
   void show_n (void)
   {
      cout << "Roll No. : "<< r_no;
      cout << "\n";
   }
};

class test:public student //Intermediate base class
{
   protected : int part1, part2;
   public :
      void get_m (int x, int y)
      {
```

MITRC

```
        part1 = x;
        part2 = y;
    }

    void show_m (void)
    {
        cout << "Marks Obtained : " << "\n"
        << "Part 1 = " << part1 << "\n"
        << "Part 2 = " << part2 << "\n";
    }
};

class sports // base for result
{
   protected : int score;
   public:
   void get_s (int s)
   {
      score = s;
   }

   void show_s (void)
   {
      cout << "Sports Weightage : " << score << "\n";
   }
};

class result : public test, public sports //Derived from test & sports
```

MITRC

```
{
   int total;
   public:
      void display(void);
};

void result::display(void)
{
   total = part1 + part2 + score;
   show_n();
   show_m();
   show_s();
   cout << "Total score : " <<total<< "\n";
};

main ( )
{
   result S1;
   S1.get_n(347) ;
   S1.get_m(30, 35);
   S1.get_s(7) ;
   S1.display() ;
}
```

MITRC

LECTURE - 11

Role of Virtual Base Class, Constructor and Destructor Execution

Virtual Base Classes

We use Single Inheritance, Multiple, Multilevel, Hybrid Inheritance. But in a situation, where we need to use multilevel, multiple, and hierarchical Inheritance.

For example, let us assume that the ***child*** has two direct base classes ***parent1*** and ***parent2*** which themselves has a common base class ***grandparent***. The child inherits the traits of ***grandparent*** via two separate paths. It can also be inherit directly as shown by the broken line. The ***grandparent*** is sometimes referred to as ***INDIRECT BASE CLASS***. Now, the inheritance by the ***child*** might cause some problems. All the public and protected members of **grandparent** are inherited into ***child*** twice, first via ***parent1*** and again via ***parent2***. So, there occurs a duplicacy which should be avoided.

The duplication of the inherited members can be avoided by making common base class as the **Virtual Base Class.**

Example-

```
#include<iostream>
#include<conio.h>
using namespace std;
class student // Base class declaration
{
   protected: int r_no;
   public:
      void get_n (int a)
      {
         r_no = a;
      }
      void show_n (void)
      {
         cout << "Roll No. " << r_no<< "\n";
      }
};
```

MITRC

```
class test : virtual public student // Virtually declared common base class 1 by using virtual keyword
{
  protected:
    int part1;
    int part2;
  public:
    void get_m (int x, int y)
    {
      part1= x; part2=y;
    }
    void show_m (void)
    {
      cout << "Marks Obtained: " << "\n";
      cout << "Part-1 = " << part1 << "\n";
      cout << "Part-2 = "<< part2 << "\n";
    }
};

class sports : public virtual student // virtually declared common base class 2 by using virtual keyword
{
  protected:
    int score;
  public:
    void get_s (int a)
    {
      score = a ;
```

MITRC

```
    }
    void show_s (void)
    {
       cout << "Sports Weightage : " <<score<< "\n";
    }
};

class result: public test, public sports //derived class
{
   private : int total ;
   public:
      void show (void) ;
};

void result::show (void)
{
   total = part1 + part2 + score ;
   show_n();
   show_m();
   show_s() ; cout << "\nTotal Score= " <<total<< "\n" ;
}

void main()
{
   result S1 ;
   S1.get_n(345);
   S1.get_m(30, 35);
   S1.get_s(7);
```

MITRC

```
   S1. show() ;
}
```

Execution of Constructor and Destructor

Construction always starts with the base class. If there are multiple base classes then, construction starts with the left most base. (If there is a virtual inheritance then it's given higher preference).

- ❑ Then the member fields are constructed. They are initialized in the order they are declared.
- ❑ Finally, the class itself is constructed.
- ❑ The order of the destructor is exactly the reverse.

Constructor

A class can include a special function called constructor, which is automatically called whenever a new object of this class is created. This constructor function must have the same name as the class and cannot have any return type. Constructors cannot be called explicitly as if they were regular member functions. They are only executed when a new object of that class is created.

Example-

```
#include <iostream>
using namespace std;
class ClassRectangle
{
   int width, height;
   public:
      ClassRectangle (int,int); //defining constructor

      int area ()
      {
         return (width * height);
      }
```

MITRC

```
};

ClassRectangle::ClassRectangle (int a, int b)
{
  width = a;
  height = b;
};

int main ()
{
  ClassRectangle recta (8, 9);
  ClassRectangle rectb (6, 3);
  cout << "Rectangle area: " << recta.area() << endl;
  cout << "Rectangle area: " << rectb.area() << endl;
  return 0;
}
```

Destructor

The destructor fulfils the opposite functionality. It is automatically called when an object is destroyed, either because its scope of existence has finished (for example, if it was defined as a local object within a function and the function ends) or because it is an object dynamically assigned and it is released using the operator delete.

The destructor must have the same name as the class but preceded with a tilde sign (~) and it must also return no value.

The use of destructors is especially suitable when an object assigns dynamic memory during its lifetime and at the moment of being destroyed we want to release the memory that the object was allocated.

Example-

```
#include <iostream>
using namespace std;
```

MITRC

```
class ClassRectangle
{
   int *width, *height;
   public:
   ClassRectangle(int,int);
   ~ClassRectangle();
   int area ()
   {
      return (*width * *height);
   }
};

ClassRectangle::ClassRectangle (int a, int b)
{
   width = new int;
   height = new int;
   *width = a;
   *height = b;
}
ClassRectangle::~ClassRectangle ()
{
   delete width;
   delete height;
   cout<<"\n Destructor executed";
}

int main ()
```

MITRC

```
{
   ClassRectangle recta (3,4);
   ClassRectangle rectb (5,6);
   cout << "Rect-A Area : " << recta.area() << endl;
   cout << "Rect-B Area : " << rectb.area() << endl;
   return 0;
}
```

MITRC

LECTURE - 12

Base Initialization using Derived Class Constructors

C++ prevents classes from initializing inherited member variables in the member initializer list of a constructor. Means, the value of a member variable can only be set in a member initializer list of a constructor belonging to the same class as the variable. But C++ gives us the ability to explicitly choose which Base class constructor will be called! We can do this by simply adding a call to the Base class constructor in the member initializer list of the derived class.

Example-

```
#include <iostream>
#include<conio.h>

using namespace std;

class alpha
{
   int x;
   public:
      alpha(int i)
      {
        x=i;
        cout << "alpha initialized \n";
      }
      void show_x(void)
      {
         cout << "x = " << x << "\n";
      }
};

class beta
{
```

MITRC

```
    float y;
    public:
        beta(float j)
        {
            y=j;
            cout << "bet initialized \n";
        }
        void show_y(void)
        {
            cout << "y = " << y << "\n";
        }
};

class gamma:public beta, public alpha
{
    int m,n;
    public:
        gamma(int a, float b, int c, int d):alpha(a), beta(b)
        {
            m=c;
            n=d;
            cout << "gamma initialized \n";
        }
        void show_mn(void)
        {
            cout <<"m = " << m <<"\n" << "n = " << n << "\n";

        }
```

MITRC

```
};

int main()
{
   gamma g(5,10.75,20,30);
   cout << "\n";
   g.show_x();
   g.show_y();
   g.show_mn();
   return 0;
}
```

In the above example constructors of alpha and beta base classes are initialized from gamma derived class.

MITRC